Neural-Lifted Sum-of-Exponentials: Learn-to-Discretize Fractional Riccati Equations for Rough Volatility

Zihan Zeng [1]

Abstract

Rough volatility models improve short-maturity option pricing but require repeatedly solving fractional Riccati–Volterra equations with singular power-law kernels. The resulting non-Markovian dependence is a major computational bottleneck in calibration and complicates end-to-end differentiation. We introduce Neural-Lifted Sum-of-Exponentials (Neural-Lifted SOE), a *learn-to-discretize* module that maps kernel parameters and a fixed computational budget to a positive sum-of-exponentials approximation. The approximation induces a low-dimensional Markovian lifting, converting the Volterra equation into a compact system of ordinary differential equations that is compatible with modern differentiable ODE solvers. By optimizing kernel-level objectives rather than price-level losses, the method yields an interpretable numerical primitive with a transparent error-propagation chain from kernel approximation to Riccati solution accuracy and downstream option prices. Neural-Lifted SOE therefore provides a reusable, auditable, and fully differentiable building block for rough volatility and affine Volterra models.

1. Introduction

Rough volatility models reconcile several empirical features of equity index options, most notably the pronounced steepening of the implied-volatility skew at short maturities (Gatheral et al., 2018). In these models, the variance dynamics exhibit fractional memory and can be expressed within the affine Volterra framework (Abi Jaber et al., 2019), with rough Heston as a canonical example (El Euch & Rosenbaum, 2019). The key analytical tool for transform-based pricing remains a Riccati characterization, but the classical Riccati ordinary differential equation of Heston (Heston, 1993) is replaced by a fractional Riccati–Volterra equation with a singular power-law kernel (Bayer et al., 2016). This loss of Markovian structure creates a practical bottleneck: calibration requires solving the same memory equation across many strikes and maturities, across Fourier frequencies in Carr–Madan style pricing, and across parameter values during optimization (Carr & Madan, 1999; Gatheral, 2006). Even in classical Markovian stochastic-volatility models, robust calibration benefits from accurate and stable Riccati solvers (Cui et al., 2017); in rough volatility the Volterra solve becomes the dominant cost. Robust numerical schemes have been developed, including fast hybrid methods for rough volatility and related Brownian semistationary models, as well as convolution-quadrature discretizations (Callegaro et al., 2021; Bennedsen et al., 2017; Lubich, 1988). Nevertheless, repeated evaluation at calibration scale and reliable end-to-end differentiation remain challenging (Ma et al., 2018; Rackauckas & Nie, 2017).

A principled approach to recovering computational tractability is *Markovian lifting*, which approximates the power-law kernel by a positive sum of exponentials and thereby converts the Volterra equation into a low-dimensional system of ordinary differential equations (Abi Jaber & El Euch, 2019; Bayer & Breneis, 2023). Classical constructions rely on hand-designed exponential-sum approximations (Beylkin & Monzón, 2005) or rational approximations specialized to rough Heston Riccati structure (Gatheral & Radoičić, 2019), with modern refinements emphasizing stability and efficiency at fixed budgets (Jiang & Zhang, 2017). In parallel, machine-learning approaches have proposed price-level surrogates to accelerate calibration (Horvath et al., 2021; Bayer & Stemper, 2018; Bayer et al., 2019), but such surrogates typically provide limited transparency about error propagation through the numerical pipeline.

We instead learn the discretization itself. We introduce **Neural-Lifted Sum-of-Exponentials (Neural-Lifted SOE)**, a *learn-to-discretize* module that maps kernel parameters and a fixed computational budget to a positive sum-of-exponentials approximation. The learned approximation induces a compact lifted ordinary differential equation system that integrates directly with differentiable solver toolchains (Chen et al., 2018; Rackauckas et al., 2019; 2020), and com-

[1]College of Economics (Finance Major), Hunan Agricultural University, Changsha, Hunan, China. Correspondence to: Zihan Zeng <zzh2489700944@outlook.com, Phone: 15700850989>.

Preprint. March 5, 2026.

plements continuous-time learning paradigms such as neural controlled differential equations and neural stochastic differential equations (Kidger et al., 2020; 2021; Liu et al., 2019). Because optimization is performed at the kernel level, we obtain a mathematically grounded error-propagation chain from kernel approximation to Riccati solution accuracy and option prices. Related microstructural motivations for rough volatility, including nearly unstable Hawkes limits, are discussed in Section 5 (Jaisson & Rosenbaum, 2015).

Our contributions are as follows:

1. **Learned positive lifting at fixed budgets.** We propose Neural-Lifted SOE, which outputs a positive sum-of-exponentials approximation for a given kernel and computational budget, inducing a low-dimensional Markovian lifting for fractional Riccati–Volterra equations (Abi Jaber & El Euch, 2019; Bayer & Breneis, 2023).

2. **Kernel-level objectives with auditability.** We optimize discretization objectives in kernel and Laplace-transform space, yielding an explicit error-propagation chain from kernel approximation to Riccati solution error and pricing error.

3. **Differentiable calibration through lifted ODEs.** The lifted representation supports stable gradients via modern adjoint-based sensitivity analysis and differentiable ODE solvers (Ma et al., 2018; Chen et al., 2018).

The remainder of this paper is organized as follows. Section 2 reviews the fractional Riccati–Volterra equation and its Laplace-measure representation. Section 3 introduces Neural-Lifted SOE and the kernel-level training objectives. Section 4 presents pricing, calibration, and ablation experiments, and the appendix provides additional bounds and kernel-only studies.

2. Preliminaries

2.1. Fractional Riccati Equation as a Volterra Convolution

We present the rough Heston fractional Riccati equation (FRE) in the form most amenable to (i) Markovian lifting and (ii) kernel-level error analysis. Fix a maturity $T > 0$ and a Fourier argument $u \in \mathbb{R}$. The (complex-valued) Riccati unknown $\psi : [0, T] \to \mathbb{C}$ solves the nonlinear Volterra convolution equation

$$\begin{aligned} \psi(t; u) &= \int_0^t K_\alpha(t-s)\, F\big(\psi(s; u); u, \theta\big)\, ds, \\ \psi(0; u) &= 0. \end{aligned} \tag{1}$$

Here θ collects model parameters and the kernel is the (fractional integration) power-law

$$K_\alpha(t) = \frac{t^{\alpha-1}}{\Gamma(\alpha)}, \qquad \alpha \in (1/2, 1). \tag{2}$$

Equivalently, (1) can be written as $\psi = I^\alpha F(\psi)$ where I^α denotes the Riemann–Liouville fractional integral (Podlubny, 1999). Under mild regularity, it is also equivalent to a (Caputo) fractional differential equation (Podlubny, 1999).

For rough Heston (El Euch & Rosenbaum, 2019), the nonlinearity is Riccati-quadratic,

$$F(\psi; u, \theta) = \frac{1}{2}(u^2 + iu) + (\rho\eta i u - \kappa)\,\psi + \frac{1}{2}\eta^2\psi^2. \tag{3}$$

Assumption 2.1 (Well-posedness and boundedness on $[0, T]$)**.** Fix $u \in \mathbb{R}$, $T > 0$, and parameters θ. Assume that the Volterra equation (1) admits a unique continuous solution on $[0, T]$ and that the solution is bounded, i.e., $\|\psi(\cdot; u)\|_{\infty,[0,T]} \le M$ for some $M > 0$. Moreover assume that $F(\cdot; u, \theta)$ is Lipschitz on the disk $\{z \in \mathbb{C} : |z| \le M\}$.

Remark 2.2 (Explicit local Lipschitz bound for rough Heston). For the rough Heston nonlinearity (3), on the disk $\{z \in \mathbb{C} : |z| \le M\}$ one has

$$\begin{aligned} |F(x; u, \theta) - F(y; u, \theta)| &\le L_M(u, \theta)\, |x - y|\,, \\ L_M(u, \theta) &:= |\rho|\ \eta\ |u| + \kappa + \eta^2 M. \end{aligned} \tag{4}$$

This follows since $\partial_\psi F(\psi; u, \theta) = (\rho\eta i u - \kappa) + \eta^2\psi$. We will use such explicit constants to state failure conditions (e.g., when stability constants blow up for large $|u|$).

Once ψ is obtained, the characteristic function is exponential-affine:

$$\begin{aligned} \phi(T; u) = \exp\Big(& iu \log S_0 + (r - q) iuT + v_0\psi(T; u) \\ & + \kappa\bar{v} \int_0^T \psi(s; u)\, ds \Big). \end{aligned} \tag{5}$$

Thus online pricing needs both $\psi(T; u)$ and $\int_0^T \psi(s; u) ds$ for many u and many T.

2.2. Completely Monotone Kernels and Laplace Measures

The power-law kernel (2) is completely monotone on $(0, \infty)$, hence (by Bernstein's theorem) admits a Laplace-transform representation by a positive measure (Schilling et al., 2012). For $\alpha \in (0, 1)$ one can write explicitly

$$\begin{aligned} K_\alpha(t) &= \frac{\sin(\pi\alpha)}{\pi} \int_0^\infty e^{-xt}\, x^{-\alpha}\, dx, \\ &\equiv \int_0^\infty e^{-xt}\, \mu_\alpha(dx), \\ \mu_\alpha(dx) &= \frac{\sin(\pi\alpha)}{\pi} x^{-\alpha} dx. \end{aligned} \tag{6}$$

The equality follows from the change of variables $y = xt$ and the reflection identity $\Gamma(\alpha)\Gamma(1-\alpha) = \pi/\sin(\pi\alpha)$.

We emphasize the numerical perspective: approximating K_α by a positive sum of exponentials is equivalent to approximating the Laplace-space measure μ_α by a positive discrete measure (Beylkin & Monzón, 2005; Jiang & Zhang, 2017).

Finally, note that the L^1-mass of K_α is explicit:

$$\|K_\alpha\|_{L^1(0,T)} = \int_0^T \frac{t^{\alpha-1}}{\Gamma(\alpha)}\,dt = \frac{T^\alpha}{\Gamma(\alpha+1)}. \tag{7}$$

This quantity appears repeatedly in well-posedness and stability conditions for Volterra equations.

The log-change of variables $x = e^s$ yields an equivalent integral over $s \in \mathbb{R}$:

$$K_\alpha(t) = \frac{\sin(\pi\alpha)}{\pi}\int_{-\infty}^{\infty} \exp\big(-te^s + (1-\alpha)s\big)\,ds. \tag{8}$$

This form isolates the singularity at $t \downarrow 0$ into an exponentially weighted integrand and is the starting point for stable quadrature.

2.3. SOE Kernels and Markovian Lifting

An m-term sum-of-exponentials (SOE) kernel is (Beylkin & Monzón, 2005; Jiang & Zhang, 2017)

$$\widehat{K}(t) = \sum_{i=1}^{m} w_i e^{-\lambda_i t}, \qquad w_i > 0,\ \lambda_i > 0. \tag{9}$$

Equivalently, $\widehat{K}$ is the Laplace transform of the discrete measure $\widehat{\mu} = \sum_{i=1}^m w_i\,\delta_{\lambda_i}$. Replacing K_α by $\widehat{K}$ in (1) yields a finite-dimensional Markovian state augmentation.

Define for each $i \in \{1,\dots,m\}$ an auxiliary state

$$y_i(t;u) := \int_0^t e^{-\lambda_i(t-s)}\,F\big(\widehat{\psi}(s;u);u,\theta\big)\,ds. \tag{10}$$

Then $\widehat{\psi}(t;u) = \sum_{i=1}^m w_i y_i(t;u)$ and, differentiating under the integral sign,

$$\begin{aligned}\dot{y}_i(t;u) &= -\lambda_i y_i(t;u) + F\big(\widehat{\psi}(t;u);u,\theta\big),\\ y_i(0;u) &= 0.\end{aligned} \tag{11}$$

To access $\int_0^T \widehat{\psi}(s;u)ds$ without storing the trajectory, we add

$$\dot{z}(t;u) = \widehat{\psi}(t;u), \qquad z(0;u) = 0, \tag{12}$$

so that $z(T;u) = \int_0^T \widehat{\psi}(s;u)ds$.

The ODE system (11)–(12) has dimension $m+1$ over $\mathbb{C}$ (or $2m+2$ after real and imaginary splitting) and is differentiable with respect to θ and to the SOE parameters $\{(w_i,\lambda_i)\}$ (Chen et al., 2018; Ma et al., 2018).

Proposition 2.3 (Exact lifting for SOE kernels)**.** *Fix $u \in \mathbb{R}$ and parameters θ. Let $\widehat{K}$ be the SOE kernel (9) and let $\widehat{\psi}$ be defined by*

$$\widehat{\psi}(t;u) = \int_0^t \widehat{K}(t-s)\,F(\widehat{\psi}(s;u);u,\theta)\,ds. \tag{13}$$

Define y_i by (10) and z by (12). Then $(y_1,\dots,y_m,z)$ satisfies the ODE system (11)–(12), and conversely any solution of the ODE system induces a solution $\widehat{\psi} = \sum_i w_i y_i$ of (13).

Proof. For notational brevity, set

$$g(s) := F(\widehat{\psi}(s;u);u,\theta). \tag{14}$$

Starting from (10), we differentiate with respect to t:

$$\begin{aligned}\frac{d}{dt}y_i(t;u) &= \frac{d}{dt}\int_0^t e^{-\lambda_i(t-s)}g(s)\,ds\\ &= g(t) + \int_0^t \frac{d}{dt}\big(e^{-\lambda_i(t-s)}\big)g(s)\,ds\\ &= g(t) - \lambda_i\int_0^t e^{-\lambda_i(t-s)}g(s)\,ds\\ &= g(t) - \lambda_i y_i(t;u),\end{aligned}$$

giving (11) with $y_i(0;u) = 0$. Since $\widehat{K}(t-s) = \sum_i w_i e^{-\lambda_i(t-s)}$, we have

$$\int_0^t \widehat{K}(t-s)g(s)\,ds = \sum_{i=1}^m w_i\,y_i(t;u) = \widehat{\psi}(t;u),$$

establishing (13). The converse follows by reversing the above identities. □

3. Methodology: Neural-Lifted SOE

We now specify (i) a strictly positive, maturity-stable parameterization of SOE kernels that can be interpreted as a quadrature rule for the Laplace measure μ_α, and (ii) an offline training procedure that learns a *budgeted discretization rule* by minimizing an analytic, kernel-level objective. The key design principle is that we learn the discretization of the kernel and measure (a reusable numerical primitive), not a surrogate mapping from parameters to prices.

3.1. Scale Separation and the Role of $\tau_{\min}$

The power-law kernel is singular at $t = 0$, and any practical solver only resolves time down to a smallest scale $t_{\min} > 0$ (e.g., the minimum step size of an ODE solver or the finest maturity used for calibration). We therefore define the approximation domain as $t \in [t_{\min}, T_{\max}]$ and introduce the dimensionless time $\tau = t/T_{\max}$ with $\tau_{\min} = t_{\min}/T_{\max}$.

Using homogeneity, for $t = T_{\max}\tau$ we have

$$K_\alpha(t) = T_{\max}^{\alpha-1}\, K_\alpha^{(1)}(\tau), \qquad K_\alpha^{(1)}(\tau) = \frac{\tau^{\alpha-1}}{\Gamma(\alpha)}. \tag{15}$$

Hence it suffices to learn an SOE approximation for $K_\alpha^{(1)}$ on $\tau \in [\tau_{\min}, 1]$; scaling back yields an approximation for any $T_{\max}$.

3.2. Positive Log-Quadrature Parameterization

Starting from the log-domain integral (8) (with t replaced by $\tau \in [\tau_{\min}, 1]$), we interpret SOE approximation as quadrature over the log-rate variable s. Fix m and choose a log-rate interval $[s_L, s_R]$ with

$$s_L = -4, \qquad s_R = \log(1/\tau_{\min}) + 4. \tag{16}$$

The heuristic is that, for a given τ, Laplace rates around $\tilde\lambda \asymp 1/\tau$ dominate the integral; the additive buffer of ± 4 spans roughly $e^{\pm 4} \approx 55$ multiplicative scales.

Given nodes $s_i \in (s_L, s_R)$ and positive bin widths $\Delta_i > 0$, the induced SOE parameters in dimensionless time are

$$\tilde\lambda_i = e^{s_i}, \qquad \tilde w_i = \frac{\sin(\pi\alpha)}{\pi}\, e^{(1-\alpha)s_i}\, \Delta_i, \tag{17}$$

which guarantees $\tilde\lambda_i > 0$ and $\tilde w_i > 0$ by construction.

Remark 3.1 (SOE as a Riemann sum for the log-domain integral). Let $G_\alpha(\tau, s) := \exp\big(-\tau e^s + (1-\alpha)s\big)$. On $\tau \in [\tau_{\min}, 1]$, the discretization (17) corresponds to the quadrature rule

$$K_\alpha^{(1)}(\tau) \approx \frac{\sin(\pi\alpha)}{\pi} \sum_{i=1}^m G_\alpha(\tau, s_i)\, \Delta_i = \sum_{i=1}^m \tilde w_i e^{-\tilde\lambda_i \tau}. \tag{18}$$

In particular, learning (s_i, Δ_i) is precisely learning an adaptive quadrature of the Laplace-measure integral.

Remark 3.2 (Truncation error from $[s_L, s_R]$). The truncation to a finite log-rate interval induces a tail error $\int_{(-\infty, s_L)\cup(s_R,\infty)} G_\alpha(\tau, s)\, ds$. For $\tau \in [\tau_{\min}, 1]$, it admits explicit bounds in terms of e^{s_L} and e^{s_R}; see Section A.1 for a complete statement and proof.

Zeroth-moment matching. We enforce that the total mass of the kernel matches exactly. First note that

$$\int_0^1 K_\alpha^{(1)}(\tau)\, d\tau = \frac{1}{\Gamma(\alpha+1)}. \tag{19}$$

For an SOE kernel $\sum_i \tilde w_i e^{-\tilde\lambda_i \tau}$,

$$\int_0^1 \sum_{i=1}^m \tilde w_i e^{-\tilde\lambda_i \tau}\, d\tau = \sum_{i=1}^m \tilde w_i\, \frac{1 - e^{-\tilde\lambda_i}}{\tilde\lambda_i}. \tag{20}$$

We therefore rescale weights by

$$c(\alpha) := \frac{1/\Gamma(\alpha+1)}{\sum_{i=1}^m \tilde w_i\, (1 - e^{-\tilde\lambda_i})/\tilde\lambda_i}, \qquad \tilde w_i \leftarrow c(\alpha)\, \tilde w_i. \tag{21}$$

This constraint stabilizes low-frequency bias without sacrificing positivity.

Scaling back to physical time. Define physical-time parameters

$$\lambda_i = \tilde\lambda_i / T_{\max}, \qquad w_i = T_{\max}^{\alpha-1} \tilde w_i. \tag{22}$$

Then $\widehat K(t) = \sum_i w_i e^{-\lambda_i t}$ approximates $K_\alpha(t)$ on $t \in [t_{\min}, T_{\max}]$.

3.3. Neural Quadrature: Learning a Budgeted Discretization Rule

We now define a learnable map

$$g_\vartheta : (\alpha, T_{\max}, \tau_{\min}, m) \mapsto \{(w_i, \lambda_i)\}_{i=1}^m \tag{23}$$

that outputs a positive and ordered set of rates and weights. Importantly, the scoring network h_ϑ does not need to depend on $T_{\max}$ because (15) makes $T_{\max}$ a pure rescaling (handled deterministically in (22)).

Inputs and shared scoring network. Let $\xi_i = (i - \frac{1}{2})/m \in (0, 1)$. We use a shared MLP $h_\vartheta : \mathbb{R}^3 \to \mathbb{R}$ to score each bin:

$$r_i = h_\vartheta\big([\alpha, \log \tau_{\min}, \xi_i]\big). \tag{24}$$

We convert scores into a probability simplex via a softmax

$$p_i = \frac{e^{r_i}}{\sum_{k=1}^m e^{r_k}}, \qquad p_i > 0,\ \sum_i p_i = 1. \tag{25}$$

From simplex to ordered log-nodes. We interpret p_i as an adaptive partition of $[s_L, s_R]$:

$$\Delta_i = (s_R - s_L)\, p_i, \qquad s_i = s_L + \sum_{k<i} \Delta_k + \tfrac{1}{2}\Delta_i. \tag{26}$$

This deterministically implies $\Delta_i > 0$ and strict ordering $s_1 < \cdots < s_m$. Mapping via (17) and applying (21) and (22) yields (w_i, λ_i).

By construction, g_ϑ is differentiable in ϑ and returns parameters satisfying positivity and ordering constraints without any projection step.

3.4. Kernel-First Training Objective

We train g_ϑ offline using only the analytic kernel, decoupled from any market data. Fix $(T_{\max}, \tau_{\min}, m)$ and sample $\alpha \sim$

Algorithm 1 Neural-Lifted SOE map g_ϑ (kernel discretizer)

Input: $\alpha \in (0,1)$, $T_{\max} > 0$, $\tau_{\min} \in (0,1)$, budget $m \in \mathbb{N}$.
Constants: $s_L = -4$, $s_R = \log(1/\tau_{\min}) + 4$.
for $i = 1$ to m **do**
  $\xi_i \leftarrow (i - 0.5)/m$, $r_i \leftarrow h_\vartheta([\alpha, \log \tau_{\min}, \xi_i])$.
end for
$p \leftarrow \mathrm{softmax}(r)$.
$\Delta_i \leftarrow (s_R - s_L)p_i$, $s_i \leftarrow s_L + \sum_{k<i} \Delta_k + 0.5\Delta_i$.
$\tilde{\lambda}_i \leftarrow \exp(s_i)$, $\tilde{w}_i \leftarrow (\sin(\pi\alpha)/\pi)\, \exp((1-\alpha)s_i)\, \Delta_i$.
$c(\alpha) \leftarrow \big(1/\Gamma(\alpha+1)\big) / \sum_i \tilde{w}_i (1 - e^{-\tilde{\lambda}_i})/\tilde{\lambda}_i$.
$\tilde{w}_i \leftarrow c(\alpha)\tilde{w}_i$, $\lambda_i \leftarrow \tilde{\lambda}_i / T_{\max}$, $w_i \leftarrow T_{\max}^{\alpha-1}\tilde{w}_i$.
Return: $\{(w_i, \lambda_i)\}_{i=1}^m$.

Algorithm 2 Offline training (kernel-first)

Input: range $[\alpha_{\min}, \alpha_{\max}]$, $(T_{\max}, \tau_{\min}, m)$, grid $\{t_j\} \subset [t_{\min}, T_{\max}]$, weights $\{\omega_j\}$.
Initialize ϑ.
repeat
  Sample mini-batch $\{\alpha_b\}$.
  Compute $(w, \lambda) = g_\vartheta(\alpha_b, T_{\max}, \tau_{\min}, m)$ using Algorithm 1.
  Evaluate $\widehat{K}_{\alpha_b}(t_j) = \sum_i w_i e^{-\lambda_i t_j}$.
  $\mathcal{L}_K \leftarrow \sum_j \omega_j \big((\widehat{K}(t_j) - K_\alpha(t_j))/K_\alpha(t_j)\big)^2$.
  Update ϑ by SGD or Adam.
until converged
Return: trained parameters ϑ.

$\mathcal{U}[\alpha_{\min}, \alpha_{\max}]$. On a log-spaced grid $t_j \in [t_{\min}, T_{\max}]$, we minimize a relative-squared error

$$\mathcal{L}_K(\vartheta) = \mathbb{E}_\alpha\Big[\sum_j \omega_j \Big(\frac{\widehat{K}_\alpha(t_j; g_\vartheta) - K_\alpha(t_j)}{K_\alpha(t_j)}\Big)^2\Big]. \quad (27)$$

The weights ω_j can be chosen to balance short and long maturities (e.g., uniform in log-time). Importantly, the mass constraint (21) is enforced inside g_ϑ, so the loss focuses on higher-order shape accuracy.

We stress the alignment with downstream theory: later error-propagation bounds are expressed in terms of $\left\|K_\alpha - \widehat{K}\right\|_{L^1(0,T)}$. On $[t_{\min}, T_{\max}]$ the kernel is bounded and strictly positive, hence controlling a relative L^2 error (appropriately weighted on a log-grid) also controls the absolute L^1 error via Cauchy–Schwarz, up to constants depending on $(\alpha, t_{\min}, T_{\max})$; see Section A.3.

3.5. Online Evaluation and Differentiable Calibration

Given model parameters θ, kernel parameter α, maturity $T \le T_{\max}$, and Fourier grid $\{u_k\}$, we: (i) compute $(w, \lambda) = g_\vartheta(\alpha, T_{\max}, \tau_{\min}, m)$, (ii) solve (11)–(12) for each u_k to obtain $\widehat{\psi}(T; u_k)$ and $z(T; u_k)$, and (iii) evaluate $\widehat{\phi}(T; u_k)$ by the affine formula (5) with ψ replaced by $\widehat{\psi}$.

The full pipeline is differentiable: $\theta \mapsto F(\cdot; u, \theta) \mapsto$ ODE solve $\mapsto \widehat{\phi} \mapsto$ Fourier inversion price. This enables gradient-based calibration; in memory-constrained regimes, one can backpropagate through the ODE solver using adjoint sensitivities (Callegaro et al., 2021).

Remark 3.3 (Complex-valued ODE implementation). In practice we solve a real ODE by stacking real and imaginary parts. For example, each $y_i \in \mathbb{C}$ becomes $(\Re y_i, \Im y_i) \in \mathbb{R}^2$ and F is evaluated via real arithmetic. This avoids relying on complex-mode support in ODE packages and makes Jacobians explicit.

3.6. Why Learn the Kernel (Not the Price)

Many deep calibration methods learn a surrogate mapping directly from parameters to prices or implied volatilities (Horvath et al., 2021). In contrast, Neural-Lifted SOE learns a structured discretization of a completely monotone kernel. The output is a mechanistic numerical scheme (an SOE kernel plus a lifted ODE) that is reusable across pricing tasks and can be audited using classical numerical analysis tools. In particular, it connects naturally to Markovian multi-factor approximations (Abi Jaber & El Euch, 2019) and admits explicit error propagation bounds (developed in later sections) of the form

$$\begin{aligned} \left\|K_\alpha - \widehat{K}\right\|_{L^1(0,T)} &\Rightarrow \left\|\psi - \widehat{\psi}\right\|_{\infty,[0,T]} \\ &\Rightarrow \left|\phi - \widehat{\phi}\right| \Rightarrow \left|C - \widehat{C}\right|. \end{aligned} \quad (28)$$

4. Experiments

We report workshop-scale experiments that require no market data. All reported statistics aggregate over 5 random seeds (0–4); the full reproduction pipeline (including figure and table staging) is scripted in `scripts/reproduce_paper_5seed.py`. Exp. 1 is a full end-to-end benchmark that traces the kernel-to-price error chain under fixed SOE budgets. Exp. 2 is a kernel-only grid that localizes where learning improves discretization quality across roughness and budgets. Together, these experiments target **Claim C3**: under a fixed SOE budget m, Neural-Lifted SOE improves the end-to-end error–speed trade-off over strong deterministic baselines, while achieving *near-real-time* per-surface runtimes (hundreds of milliseconds on CPU), i.e., aligning the speed dimension with SOTA-scale latency requirements for calibration pipelines.

4.1. End-to-End Error–Speed (Exp.1)

Setup and data. We generate synthetic rough-Heston surfaces for three roughness regimes $\alpha \in \{0.65, 0.75, 0.85\}$. For each α we evaluate 20 parameter sets: 5 fixed nominal sets and 15 random draws from $\kappa \sim \mathcal{U}[0.5, 4.0]$, $\bar{v} \sim \mathcal{U}[0.02, 0.12]$, $\eta \sim \mathcal{U}[0.2, 1.2]$, $\rho \sim \mathcal{U}[-0.95, -0.1]$, and $v_0 \sim \mathcal{U}[0.02, 0.12]$. We set $S_0 = 1$ and $r = q = 0$ (forward measure). Maturities are $T \in \{0.05, 0.10, 0.25, 0.50, 1.00\}$ and log-moneyness $k = \log(K/F_T)$ is uniform on $[-0.35, 0.35]$ with 41 points, yielding 205 options per surface. Overall we price 60 surfaces.

Discretizers and budgets. We compare three discretizers under the same budget m: (i) **Uniform** log-bins ($p_i \equiv 1/m$ in Algorithm 1), (ii) a **Deterministic** multi-factor SOE baseline that chooses log-bin widths by an equal-mass partition of the averaged log-domain integrand (no learning; still outputs (w, λ) via Algorithm 1 and the same moment matching), and (iii) the learned discretizer g_ϑ. Budgets are $m \in \{8, 16, 32\}$. Each budget uses its own checkpoint trained with the kernel-only loss in Section 3.4 using `nls_soe.train.train_kernel`: $\alpha \sim \mathcal{U}[0.55, 0.99]$, $\tau_{\min} = 10^{-4}$, log-time grid size 256, Adam with lr $2 \cdot 10^{-3}$ for 20,000 steps and batch size 256. We use the same log-domain margin $s_{\text{margin}} = 12$ at train and test time.

Lifted ODE and pricing. For each $(\alpha, m, \text{method})$ we precompute (w, λ) and solve the lifted ODE on $[0, T_{\max}]$ with $T_{\max} = 1.0$, $\tau_{\min} = 10^{-4}$ (so $t_{\min} = 10^{-4}$), using exponential Euler with 2048 uniform steps and trapezoid integration for z (`z_scheme=trap`, corrector off). We integrate once to $T_{\max}$ and record states at the maturity indices. Fourier pricing uses the Lewis formula with $u \in [0, 60]$ on a 512-point grid, evaluated at the shifted argument $u - \frac{i}{2}$ and integrated by the trapezoid rule. Implied volatilities are obtained by bisection with bounds $[10^{-4}, 5]$.

Reference and metrics. We compute reference prices and implied volatilities using a high-accuracy uniform SOE with a wider truncation margin $s^{\text{ref}}_{\text{margin}} = 18$, $m_{\text{ref}} = 64$, 4096 time steps, and float64 arithmetic (on the same u, k, and T grids). Kernel diagnostics are evaluated on a 512-point log-time grid and include relative L^2, relative L^∞, absolute L^1, and mass error. Per surface, we log the price RMSE and maximum error, the implied-volatility RMSE and maximum error (bp), the maximum $|\psi - \psi_{\text{ref}}|$ and $|\phi - \phi_{\text{ref}}|$, the runtime per surface, and a failure flag for any non-finite output. Figure 1 summarizes kernel L^1 error vs budget and IV RMSE vs runtime (panels a–b), a kernel-to-IV diagnostic scatter plot (panel c), and the Exp. 2 heatmap over (m, α) (panel d); the Exp. 2 slices versus α are given in Figure 2. Table 1 summarizes median kernel error, runtime, IV RMSE, and failure rate (medians and interquartile ranges are reported in the figure panels).

Table 1. Exp.1 summary (medians over surfaces): kernel L^1 error, end-to-end runtime, IV RMSE, and failure rate, comparing uniform vs deterministic vs learned SOE discretization.

Method	m	$\|K - K_m\|_{L^1}$	time (ms)	IV RMSE (bp)	fail %
Learned	8	0.020	269.1	330.6	2.0
Learned	16	0.034	242.9	292.1	2.3
Learned	32	0.009	272.6	144.9	2.3
Deterministic	8	0.020	269.6	479.8	2.7
Deterministic	16	0.015	242.3	357.7	2.3
Deterministic	32	0.010	273.2	257.0	2.3
Uniform	8	0.066	268.5	919.9	2.7
Uniform	16	0.014	242.2	328.3	2.3
Uniform	32	0.010	269.8	234.8	2.3

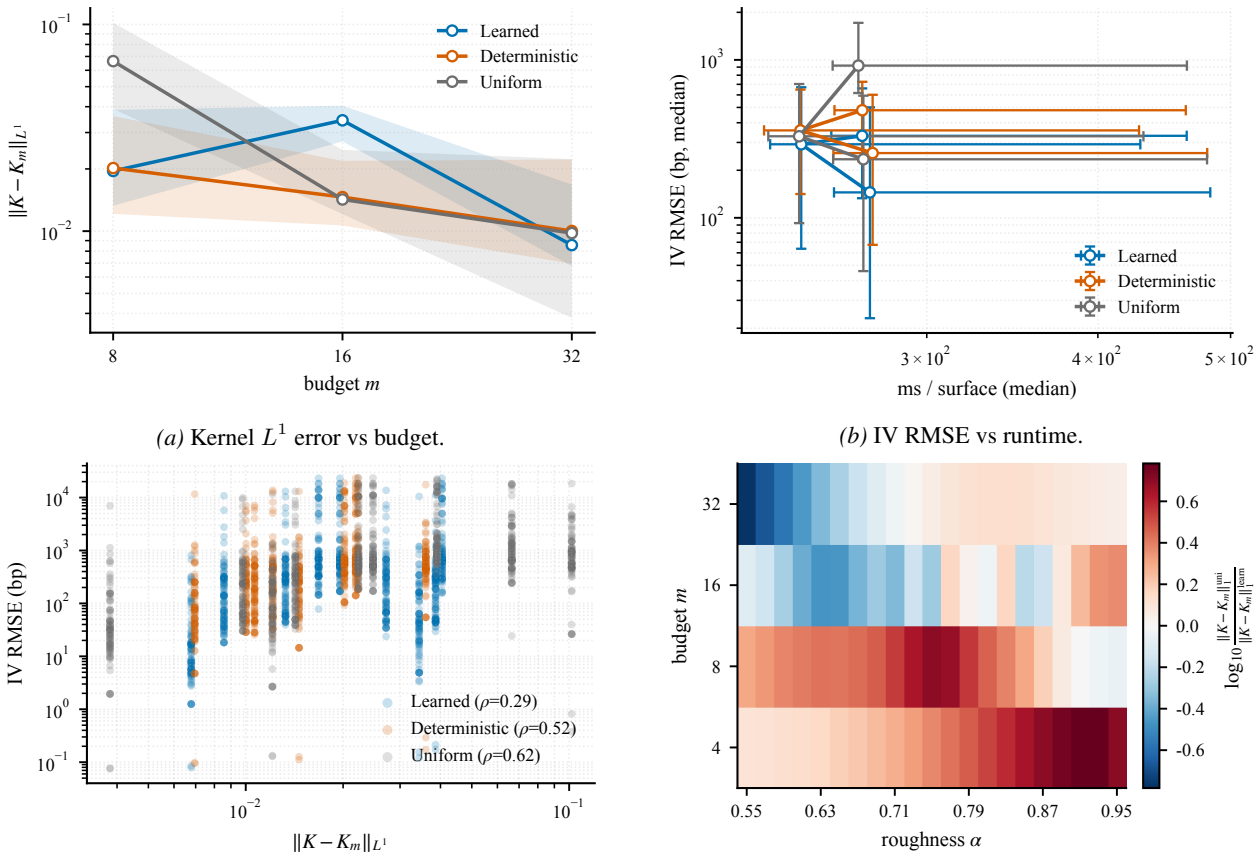
Table 2. Exp.1 runtime breakdown (ms) for a representative rough-Heston surface evaluation on CPU: kernel discretization, lifted ODE solve, and Fourier inversion plus implied-volatility bisection (medians over repeated runs).

Method	m	kernel (ms)	ODE (ms)	Fourier+IV (ms)	total (ms)
Learned	8	0.23	229.62	8.91	241.05
Learned	16	0.26	216.60	9.57	224.63
Learned	32	0.24	253.10	10.31	263.16
Deterministic	8	0.54	240.77	9.03	249.67
Deterministic	16	0.47	218.77	10.21	230.10
Deterministic	32	0.46	244.85	8.48	252.95
Uniform	8	0.17	239.09	9.51	248.31
Uniform	16	0.16	221.47	10.18	232.07
Uniform	32	0.16	240.45	9.84	250.87

Results. Across budgets, the learned discretizer improves the downstream accuracy at comparable runtime. In Table 1, learned reduces the median IV RMSE from 920 bp to 331 bp at $m = 8$ and from 235 bp to 145 bp at $m = 32$, while maintaining low failure rates ($\leq 2.3\%$).

Runtime decomposition. To make the near real-time claim concrete, we decompose the wall-clock time of a single end-to-end surface evaluation into: (i) kernel discretization (uniform, deterministic, and learned), (ii) the lifted ODE solve, and (iii) Fourier inversion together with implied-volatility back-out. Table 2 reports median timings (ms) for a representative (α, θ) on CPU under the same grids as Exp. 1. These measurements explain why the end-to-end times in Table 1 are on the order of a few hundred milliseconds: the lifted ODE solve dominates, while kernel evaluation is sub-millisecond and the Fourier+IV stage is single-digit milliseconds.

Appendix A.6 reports a kernel-only (α, m) grid study to isolate discretization error (panel d of Figure 1 and Figure 2); learning helps most at small budgets and in rough regimes (smaller α).



(a) Kernel L^1 error vs budget.

(b) IV RMSE vs runtime.

(c) Kernel L^1 error vs IV RMSE.

(d) Exp. 2 heatmap over (m, α).

Figure 1. Experiments: (a) kernel L^1 error vs budget and (b) IV RMSE vs runtime for Exp. 1; (c) diagnostic scatter plot (kernel L^1 error vs IV RMSE); (d) Exp. 2 kernel-only error grid improvement, visualized as $\log_{10}(\|K - K_m\|_1^{\text{uni}} / \|K - K_m\|_1^{\text{learn}})$ (positive values favor learning).

5. Related Work

The rough Heston characteristic function and fractional Riccati structure are due to El Euch & Rosenbaum (2019), and affine Volterra processes formalize the transform machinery underpinning such models (Abi Jaber et al., 2019). Deterministic Markovian multi-factor approximations and fast hybrid schemes provide strong numerical baselines for solving FREs (Abi Jaber & El Euch, 2019; Callegaro et al., 2021), while rational approximations offer extremely fast kernels specialized to rough-Heston-type Riccati structure (Gatheral & Radoičić, 2019).

Kernel approximation. Approximating power-law kernels by exponential sums is classical (Beylkin & Monzón, 2005), and modern work emphasizes stable SOE representations for fast fractional and Volterra evaluation (Jiang & Zhang, 2017); we instead learn a *budgeted discretization rule* that outputs positive SOE parameters as a differentiable module.

Machine learning for derivative pricing and calibration. Unlike price-level surrogates for calibration (Horvath et al., 2021; Bayer & Stemper, 2018; Bayer et al., 2019), we learn a structured numerical primitive based on kernel discretization and Markovian lifting that is reusable and auditable.

Differentiable simulation and scientific machine learning. By converting the Volterra equation into a lifted ODE system, Neural-Lifted SOE plugs into differentiable ODE toolchains (Chen et al., 2018; Ma et al., 2018; Rackauckas & Nie, 2017) while retaining explicit error accounting.

6. Limitations and Outlook

Our evaluation is restricted to synthetic rough-Heston pricing and does not include real market calibration or other affine Volterra models. The neural discretizer is trained offline; quantifying amortization and transfer across maturities, kernels, and model families remains future work.

7. Conclusion

We introduced Neural-Lifted SOE, a learn-to-discretize kernel module that produces positive SOE kernels and an induced lifted ODE for fractional Riccati equations. The method targets the computational primitive (kernel discretization) rather than the end-product (prices), enabling reusability, interpretability, and explicit error control.

Acknowledgements

Do not include acknowledgements in the initial version submitted for blind review.

Impact Statement

This paper presents work whose goal is to advance numerical methods for differentiable model-based pricing and calibration. We do not anticipate direct negative societal consequences from this line of research beyond those generally associated with the use of financial models.

References

Abi Jaber, E. and El Euch, O. Multi-factor approximation of rough volatility models. *SIAM Journal on Financial Mathematics*, 2019. doi: 10.1137/18M1170236.

Abi Jaber, E., Larsson, M., and Pulido, S. Affine Volterra processes. *The Annals of Applied Probability*, 2019. doi: 10.1214/18-AAP1462.

Bayer, C. and Breneis, S. Markovian approximations of stochastic Volterra equations with completely monotone kernels. *Quantitative Finance*, 2023. doi: 10.1080/14697688.2022.2139193.

Bayer, C. and Stemper, B. Deep calibration of rough stochastic volatility models. arXiv:1810.03399, 2018.

Bayer, C., Friz, P., and Gatheral, J. Pricing under rough volatility. *Quantitative Finance*, 16(6):887–904, 2016. doi: 10.1080/14697688.2015.1099717.

Bayer, C., Horvath, B., Muguruza, A., Stemper, B., and Tomas, M. On deep calibration. arXiv:1908.08806, 2019.

Bennedsen, M., Lunde, A., and Pakkanen, M. S. Hybrid scheme for brownian semistationary processes. *Finance and Stochastics*, 2017. doi: 10.1007/s00780-017-0335-5.

Beylkin, G. and Monzón, L. On approximation of functions by exponential sums. *Applied and Computational Harmonic Analysis*, 19(1):17–48, 2005. doi: 10.1016/j.acha.2005.01.002.

Brunner, H. *Collocation Methods for Volterra Integral and Related Functional Differential Equations*. Cambridge University Press, 2004.

Callegaro, G., Grasselli, M., and Pagès, G. Fast hybrid schemes for fractional riccati equations (rough Heston). *Mathematics of Operations Research*, 2021. doi: 10.1287/moor.2020.1054.

Carr, P. and Madan, D. B. Option valuation using the fast Fourier transform. *Journal of Computational Finance*, 2 (4):61–73, 1999.

Chen, T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018.

Cui, Z., del Baño Rollin, X., and Germano, G. Fast and accurate calibration of the Heston option pricing model. *European Journal of Operational Research*, 263(2):625–638, 2017. doi: 10.1016/j.ejor.2017.05.018.

El Euch, O. and Rosenbaum, M. The characteristic function of rough Heston models. *Mathematical Finance*, 2019. doi: 10.1111/mafi.12173.

Gatheral, J. *The Volatility Surface: A Practitionerś Guide*. Wiley, 2006. ISBN 9780471792514.

Gatheral, J. and Radoičić, R. Rational approximation of the rough Heston solution. *International Journal of Theoretical and Applied Finance*, 22(3):1950010, 2019. doi: 10.1142/S0219024919500109.

Gatheral, J., Jaisson, T., and Rosenbaum, M. Volatility is rough. *Quantitative Finance*, 18(6):933–949, 2018. doi: 10.1080/14697688.2017.1393551.

Heston, S. L. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2): 327–343, 1993. doi: 10.1093/rfs/6.2.327.

Horvath, B., Muguruza, A., and Tomas, M. Deep learning volatility: A deep neural network perspective on pricing and calibration. *Quantitative Finance*, 2021. doi: 10.1080/14697688.2020.1817974.

Jaisson, T. and Rosenbaum, M. Limit theorems for nearly unstable hawkes processes. *The Annals of Applied Probability*, 2015. doi: 10.1214/15-AAP1164.

Jiang, S. and Zhang, J. Fast evaluation of the Caputo fractional derivative and its applications to fractional diffusion equations: A second order scheme. *Communications in Computational Physics*, 21(3):650–678, 2017. doi: 10.4208/cicp.OA-2015-0204.

Kidger, P., Morrill, J., Foster, J., and Lyons, T. Neural controlled differential equations for irregular time series. In *Advances in Neural Information Processing Systems*, 2020.

Kidger, P., Foster, J., Li, X., Oberhauser, H., and Lyons, T. Neural SDEs as infinite-dimensional GANs. arXiv:2102.03657, 2021.

Liu, X., Li, X., Chen, T. Q., Duvenaud, D. K., et al. Neural stochastic differential equations. arXiv:1906.02355, 2019.

Lubich, C. Convolution quadrature and discretized operational calculus. I. *Numerische Mathematik*, 52:129–145, 1988. doi: 10.1007/BF01398686.

Ma, Y., Gowda, S., Anantharaman, R., et al. A comparison of automatic differentiation and continuous sensitivity analysis for derivatives of differential equation solutions. arXiv:1812.01892, 2018.

Podlubny, I. *Fractional Differential Equations*. Academic Press, San Diego, 1999. ISBN 9780125588409.

Rackauckas, C. and Nie, Q. Differentialequations.jl–a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017. doi: 10.5334/jors.151.

Rackauckas, C., Ma, Y., Noack, A., Dixit, V., Guo, X., et al. DiffEqFlux.jl–a julia library for neural differential equations. arXiv:1902.02376, 2019.

Rackauckas, C., Ma, Y., Martensen, J., Warner, C., Zubov, K., Supekar, R., Skinner, D., Ramadhan, A., Edelman, A., et al. Universal differential equations for scientific machine learning. arXiv:2001.04385, 2020.

Schilling, R. L., Song, R., and Vondraček, Z. *Bernstein Functions: Theory and Applications*. De Gruyter, 2 edition, 2012. ISBN 9783110269338.

A. Proofs and Additional Bounds

This appendix collects self-contained proofs for the auxiliary bounds invoked in Section 3. Throughout, fix $\alpha \in (0,1)$, a smallest resolved scale $\tau_{\min} \in (0,1)$, and define

$$G_\alpha(\tau, s) := \exp\big(-\tau e^s + (1-\alpha)s\big), \quad (\tau, s) \in [\tau_{\min}, 1] \times \mathbb{R}. \tag{29}$$

A.1. Tail Bounds for the Log-Domain Integral

Recall the representation

$$K_\alpha^{(1)}(\tau) = \frac{\sin(\pi\alpha)}{\pi} \int_{-\infty}^{\infty} G_\alpha(\tau, s)\, ds. \tag{30}$$

Let $c_\alpha := \frac{\sin(\pi\alpha)}{\pi}$. Given a truncation interval $[s_L, s_R]$, define the tail remainder

$$R_{\text{tail}}(\tau; s_L, s_R) := \int_{(-\infty, s_L)\cup(s_R, \infty)} G_\alpha(\tau, s)\, ds. \tag{31}$$

Lemma A.1 (Lower- and upper-tail bounds)**.** *For any $\tau \in [\tau_{\min}, 1]$ and any $s_L < s_R$, one has*

$$R_{\text{tail}}(\tau; s_L, s_R) \le \underbrace{\frac{e^{(1-\alpha)s_L}}{1-\alpha}}_{\textit{lower tail}} + \underbrace{\frac{e^{-\tau_{\min} e^{s_R}}}{\tau_{\min}}\, e^{-\alpha s_R}}_{\textit{upper tail}}. \tag{32}$$

Consequently, the truncated log-domain integral satisfies

$$\left| K_\alpha^{(1)}(\tau) - c_\alpha \int_{s_L}^{s_R} G_\alpha(\tau, s)\, ds \right| \le c_\alpha R_{\text{tail}}(\tau; s_L, s_R).$$

Proof. We bound the two tails separately.

Lower tail $(-\infty, s_L)$**.** Since $-\tau e^s \le 0$, we have $G_\alpha(\tau, s) \le e^{(1-\alpha)s}$ for all $s \in \mathbb{R}$. Therefore

$$\int_{-\infty}^{s_L} G_\alpha(\tau, s)\, ds \le \int_{-\infty}^{s_L} e^{(1-\alpha)s}\, ds = \frac{e^{(1-\alpha)s_L}}{1-\alpha}. \tag{33}$$

Upper tail (s_R, ∞)**.** Substitute $x = e^s$ (so $ds = dx/x$) to obtain

$$\int_{s_R}^{\infty} G_\alpha(\tau, s)\, ds = \int_{e^{s_R}}^{\infty} e^{-\tau x} x^{-\alpha}\, dx. \tag{34}$$

Using $\tau \ge \tau_{\min}$ and $x \ge e^{s_R}$ gives

$$\int_{e^{s_R}}^{\infty} e^{-\tau x} x^{-\alpha}\, dx \le e^{-\tau_{\min} e^{s_R}} \int_{e^{s_R}}^{\infty} x^{-\alpha}\, dx. \tag{35}$$

This bound is not finite for $\alpha \in (0,1)$, so we instead use $x^{-\alpha} \le e^{-\alpha s_R} x^0$ for $x \ge e^{s_R}$ and integrate the exponential tail:

$$\begin{aligned} \int_{e^{s_R}}^{\infty} e^{-\tau x} x^{-\alpha}\, dx &\le e^{-\alpha s_R} \int_{e^{s_R}}^{\infty} e^{-\tau x}\, dx \\ &= e^{-\alpha s_R}\, \frac{e^{-\tau e^{s_R}}}{\tau} \\ &\le \frac{e^{-\tau_{\min} e^{s_R}}}{\tau_{\min}}\, e^{-\alpha s_R}. \end{aligned} \tag{36}$$

Combining the two tail bounds yields (32). □

A.2. Moment Matching and Control of Kernel Mass

Lemma A.2 (Exact mass matching under scaling)**.** *Let $\tilde{\widehat{K}}(\tau) = \sum_{i=1}^m \tilde{w}_i e^{-\tilde{\lambda}_i \tau}$ on $\tau \in [0,1]$. Define $c(\alpha)$ by (21) and rescale $\tilde{w}_i \leftarrow c(\alpha)\tilde{w}_i$. Then*

$$\int_0^1 \tilde{\widehat{K}}(\tau)\, d\tau = \int_0^1 K_\alpha^{(1)}(\tau)\, d\tau = \frac{1}{\Gamma(\alpha+1)}. \tag{37}$$

Moreover, defining the physical-time kernel by (22),

$$\widehat{K}(t) = \sum_{i=1}^m w_i e^{-\lambda_i t}, \qquad t \in [0, T_{\max}], \tag{38}$$

yields exact matching of the L^1-mass on $[0, T_{\max}]$:

$$\int_0^{T_{\max}} \widehat{K}(t)\, dt = \int_0^{T_{\max}} K_\alpha(t)\, dt = \frac{T_{\max}^\alpha}{\Gamma(\alpha+1)}. \tag{39}$$

Proof. The first identity is exactly how $c(\alpha)$ is defined. For the physical-time scaling, substitute $t = T_{\max}\tau$:

$$\begin{aligned} \int_0^{T_{\max}} \widehat{K}(t)\, dt &= \int_0^{T_{\max}} \sum_{i=1}^m T_{\max}^{\alpha-1} \tilde{w}_i \\ &\qquad \exp\big(-(\tilde{\lambda}_i / T_{\max})t\big)\, dt \\ &= T_{\max}^\alpha \int_0^1 \tilde{\widehat{K}}(\tau)\, d\tau. \end{aligned}$$

By the first part, this equals $T_{\max}^\alpha / \Gamma(\alpha+1)$, which coincides with (7). □

A.3. From Relative L^2 Control to L^1 Control

This section formalizes the statement after (27). Let $[a, b] = [t_{\min}, T_{\max}]$ and assume $0 < a < b < \infty$.

Lemma A.3 (Cauchy–Schwarz conversion)**.** *Let K be a strictly positive function on $[a, b]$ and let $\widehat{K}$ be any approximation. Then*

$$\begin{aligned} \left\| \widehat{K} - K \right\|_{L^1(a,b)} &\le \Big(\int_a^b K(t)^2\, dt \Big)^{1/2} \\ &\quad \cdot \Big(\int_a^b \Big(\frac{\widehat{K}(t) - K(t)}{K(t)} \Big)^2 dt \Big)^{1/2}. \end{aligned} \tag{40}$$

In particular, for the power-law kernel $K = K_\alpha$ one has

$$\int_a^b K_\alpha(t)^2\,dt = \frac{1}{\Gamma(\alpha)^2}\,\frac{b^{2\alpha-1}-a^{2\alpha-1}}{2\alpha-1}, \qquad \alpha > \frac{1}{2}. \tag{41}$$

Proof. Write

$$\left|\widehat{K}(t) - K(t)\right| = K(t)\,\left|(\widehat{K}(t) - K(t))/K(t)\right|. \tag{42}$$

Apply Cauchy–Schwarz on $[a, b]$ to obtain (40). For (41), substitute $K_\alpha(t) = t^{\alpha-1}/\Gamma(\alpha)$ and integrate. □

A.4. Error Propagation from Kernel to Price

This section records a basic (and deliberately conservative) error-propagation bound that links kernel approximation error to the final option price. Related stability arguments for nonlinear Volterra equations can be found, e.g., in Brunner (2004).

Theorem A.4 (Stability of nonlinear Volterra equations under kernel perturbations)**.** *Fix $T > 0$ and a Fourier argument $u \in \mathbb{R}$. Let $K_\alpha, \widehat{K} \in L^1(0,T)$ and let $\psi, \widehat{\psi} : [0,T] \to \mathbb{C}$ solve the Volterra equations*

$$\psi = K_\alpha * F(\psi), \qquad \widehat{\psi} = \widehat{K} * F(\widehat{\psi}), \tag{43}$$

where $(K * g)(t) := \int_0^t K(t-s)g(s)\,ds$ and $F(\cdot; u, \theta)$ is the rough-Heston Riccati nonlinearity (3). Assume there exists $M > 0$ such that $\|\psi(\cdot; u)\|_{\infty,[0,T]} \le M$ and $\left\|\widehat{\psi}(\cdot; u)\right\|_{\infty,[0,T]} \le M$. Define

$$B_M(u,\theta) := \sup_{|z|\le M} |F(z; u, \theta)|,$$
$$L_M(u,\theta) := \sup_{\substack{|x|\le M\\ |y|\le M\\ x\ne y}} \frac{|F(x; u,\theta) - F(y; u,\theta)|}{|x-y|}. \tag{44}$$

If $L_M(u,\theta)\,\|K_\alpha\|_{L^1(0,T)} < 1$, then

$$\left\|\psi(\cdot; u) - \widehat{\psi}(\cdot; u)\right\|_{\infty,[0,T]} \le \frac{B_M(u,\theta)}{1 - L_M(u,\theta)\,\|K_\alpha\|_{L^1(0,T)}} \cdot \left\|K_\alpha - \widehat{K}\right\|_{L^1(0,T)}. \tag{45}$$

Proof. Subtract the two equations in (43):

$$\psi - \widehat{\psi} = (K_\alpha - \widehat{K}) * F(\widehat{\psi}) + K_\alpha * \big(F(\psi) - F(\widehat{\psi})\big).$$

Taking sup norms and using Young's inequality and the definitions of B_M and L_M yields

$$\left\|\psi - \widehat{\psi}\right\|_{\infty,[0,T]} \le \left\|K_\alpha - \widehat{K}\right\|_{L^1(0,T)} B_M(u,\theta) + \|K_\alpha\|_{L^1(0,T)}\, L_M(u,\theta) \cdot \left\|\psi - \widehat{\psi}\right\|_{\infty,[0,T]}.$$

Rearranging gives (45). □

Proposition A.5 (Explicit constants for rough Heston and Fourier pricing)**.** *Under the assumptions of Theorem A.4, for rough Heston one may take the explicit Lipschitz bound $L_M(u,\theta) = |\rho|\,\eta\,|u| + \kappa + \eta^2 M$ from Equation (4) and the crude magnitude bound*

$$B_M(u,\theta) \le \frac{1}{2}(|u|^2 + |u|) + (|\rho|\,\eta\,|u| + \kappa)\,M + \frac{1}{2}\eta^2 M^2. \tag{46}$$

Moreover, since $\|K_\alpha\|_{L^1(0,T)} = T^\alpha/\Gamma(\alpha+1)$ by (7), the stability condition reads

$$L_M(u,\theta)\,\frac{T^\alpha}{\Gamma(\alpha+1)} < 1. \tag{47}$$

Let $\phi(T; u)$ and $\widehat{\phi}(T; u)$ be the rough-Heston characteristic functions computed from ψ and $\widehat{\psi}$ via (5). Then, for each fixed $u \in \mathbb{R}$,

$$\left|\phi(T;u) - \widehat{\phi}(T;u)\right| \le \exp\big(|v_0|\,M + |\kappa\bar{v}|\,T\,M\big) \cdot \big(|v_0| + |\kappa\bar{v}|\,T\big) \cdot \left\|\psi(\cdot;u) - \widehat{\psi}(\cdot;u)\right\|_{\infty,[0,T]}. \tag{48}$$

Finally, consider any Fourier-based pricing approximation of the form $\widehat{C} = \sum_{k=1}^{n_u} \omega_k\,\widehat{\phi}(T; u_k)$ for some complex weights $\{\omega_k\}$ (e.g., a quadrature rule applied to the Lewis integral). Then

$$\left|C - \widehat{C}\right| \le \sum_{k=1}^{n_u} |\omega_k|\,\left|\phi(T;u_k) - \widehat{\phi}(T;u_k)\right|, \tag{49}$$

so combining (45) and (48) yields an explicit (though loose) kernel-to-price bound in terms of $\left\|K_\alpha - \widehat{K}\right\|_{L^1(0,T)}$.

A.5. Additional Exp. 1 Diagnostics

The diagnostic scatter plot (kernel L^1 error versus IV RMSE) is shown in panel (c) of Figure 1.

A.6. Kernel Error Grid (Exp.2)

To isolate the *learn-to-discretize* component, we measure pure kernel approximation error over a dense (α, m) grid. We approximate the power-law kernel $K_\alpha(t) = t^{\alpha-1}/\Gamma(\alpha)$ on $[t_{\min}, t_{\max}]$ with $t_{\max} = 1$ and $\tau_{\min} = t_{\min}/t_{\max} = 10^{-4}$, using a fixed log-domain margin $s_{\text{margin}} = 12$. Budgets are $m \in \{4, 8, 16, 32\}$. We compare the learned discretizer against a uniform log-bin baseline (same positivity, ordering, and zeroth-moment matching; only the bin density differs), and report $\|K_\alpha - \widehat{K}\|_{L^1(t_{\min},t_{\max})}$.

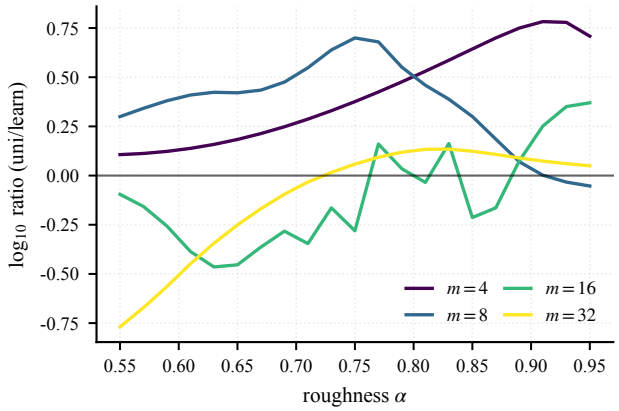


Figure 2. Exp. 2 slices versus α.

Table 3. Exp. 2 summary (medians over α): kernel L^1 error for uniform vs learned discretization and the median log-ratio improvement.

m	med $\|K - K_m\|_{L^1}^{\text{uni}}$	med $\|K - K_m\|_{L^1}^{\text{learn}}$	med $\log_{10} \frac{\text{uni}}{\text{learn}}$
4	0.163	0.068	0.38
8	0.066	0.029	0.36
16	0.016	0.023	-0.17
32	0.010	0.009	0.06

Results. Figures 1d and 2 visualizes the improvement as $\log_{10}(\|K - K_m\|_1^{\text{uni}} / \|K - K_m\|_1^{\text{learn}})$, where positive values indicate a gain from learning. The gain concentrates at small budgets and in rough regimes (smaller α), consistent with the intuition that a learned quadrature is most helpful when m is tight. Table 3 summarizes medians over α: at $m = 4$ learning reduces the median L^1 error from 0.163 to 0.068 ($\approx 2.4\times$), and at $m = 8$ from 0.066 to 0.029 ($\approx 2.3\times$). For $m \geq 16$ the gap narrows (and at $m = 16$ the uniform baseline slightly wins on the median L^1 metric), indicating saturation of kernel accuracy at moderate budgets.